

Perlmutter 101: Interactive Jobs, CPU vs GPU, and Slurm

Perlmutter is a supercomputer managed by the **Slurm** job scheduler. You should generally **not run heavy computation directly on the login node**. The login node is mainly for editing files, checking data, submitting jobs, and lightweight commands. Heavy computation should run on allocated CPU or GPU compute nodes.

1. Interactive Job vs. Batch Slurm Job

There are two common ways to use compute resources on Perlmutter.

Interactive Job

An interactive job gives you a compute node that you can work on directly in the terminal.

Use it when you are:

- developing or debugging code;
- checking whether a script runs;
- inspecting data;
- testing GPU availability;
- running a relatively short experiment where you want to see the output immediately.

You request an interactive allocation with **salloc**. Once Slurm grants the allocation, you receive access to a compute node and can run commands interactively. [\[cite turn111843search2\]](#)

Conceptually:

```
Login node
  ↓
salloc
  ↓
Slurm gives you a compute node
  ↓
You work interactively on that node
```

Batch Slurm Job

A batch job is used when you already know what program you want to run.

Instead of staying in the terminal and manually running commands, you write a Slurm script containing the resource requirements and commands, and submit it with:

```
sbatch my_job.slurm
```

Slurm places the job in the queue and runs it when the requested resources become available.

[\[cite turn111843search1 turn111843search5\]](#)

Conceptually:

```
Write job script
↓
sbatch job.slurm
↓
Slurm queue
↓
Compute node becomes available
↓
Job runs automatically
```

For our research tasks, a useful rule is:

```
Develop/debug/test → interactive job
Long or repeatable experiment → batch Slurm job
```

2. CPU Nodes vs. GPU Nodes

Perlmutter has separate **CPU-only nodes** and **GPU nodes**.

CPU Node

A CPU-only node contains:

- 2 AMD EPYC 7763 CPUs;
- 128 physical CPU cores total;
- 512 GB system memory;
- no GPU.

CPU nodes are appropriate for things such as:

- preprocessing NetCDF files;
- regridding;
- calculating statistics;
- moving/organizing data;
- CPU-based scientific models;
- scripts that do not use CUDA/PyTorch/JAX GPU acceleration.

GPU Node

A Perlmutter GPU node contains:

- 1 AMD EPYC 7763 CPU;
- 64 physical CPU cores;
- 4 NVIDIA A100 GPUs;
- 256 GB system memory;

- either 40 GB or 80 GB GPU memory per A100 depending on the node type.

GPU nodes should be used for GPU-accelerated workloads such as:

- PyTorch model training;
- JAX;
- TensorFlow;
- NeuralGCM / ACE-type neural weather models;
- other CUDA-based workloads.

Do not request a GPU node just because a task is computationally expensive. If the program does not actually use a GPU, use a CPU node.

3. Requesting an Interactive CPU Job

A basic one-node interactive CPU allocation is:

```
salloc \  
  --nodes 1 \  
  --qos interactive \  
  --time 01:00:00 \  
  --constraint cpu \  
  --account <PROJECT_ACCOUNT>
```

For example:

```
salloc -N 1 -q interactive -t 01:00:00 -C cpu -A <PROJECT_ACCOUNT>
```

Replace **<PROJECT_ACCOUNT>** with the NERSC project allocation you have been given.

Once the allocation is granted, Slurm will report the compute node assigned to you.

The official NERSC example uses the same **salloc** structure for CPU interactive jobs.

4. Requesting an Interactive GPU Job

For one GPU node, the basic NERSC syntax is:

```
salloc \  
  --nodes 1 \  
  --qos interactive \  
  --time 01:00:00 \  
  --constraint gpu \  
  --gpus 4 \  
  --account <GPU_PROJECT_ACCOUNT>
```

NERSC's GPU interactive allocation example requests all four GPUs on the node.

If your project/job setup allows requesting fewer GPUs and you only need one GPU, you may instead request the appropriate GPU count, for example:

```
salloc \  
  --nodes 1 \  
  --qos interactive \  
  --time 01:00:00 \  
  --constraint gpu \  
  --gpus 1 \  
  --account <GPU_PROJECT_ACCOUNT>
```

The GPU allocation account normally uses the project's GPU allocation name, which NERSC documents as ending in **_g**, for example:

```
m1234_g
```

GPU and CPU allocations are accounted for separately.

After receiving a GPU allocation, verify that the GPU is visible:

```
nvidia-smi
```

If launching work through **srun**, remember that NERSC requires the GPU resource to be explicitly requested in the **srun** command as well; otherwise CUDA may not see the GPU.

For example:

```
srun --gpus 1 python your_script.py
```

5. How to Check Your Jobs

To see your current Slurm jobs:

```
squeue -u $USER
```

Typical states include:

```
R    = running
PD   = pending
```

For an interactive allocation, once the node is granted, you can also check where you are running with:

```
hostname
```

A Perlmutter compute node normally has a name such as:

```
nid001234
```

6. Ending an Interactive Job

When you are finished, exit the interactive shell:

```
exit
```

Do not keep an interactive allocation running unnecessarily, because the requested resources remain allocated to the project while the job exists.

You can also inspect your jobs with:

```
squeue -u $USER
```

and cancel a particular job if necessary:

```
scancel <JOB_ID>
```

7. Which One Should You Use?

For our project, use this rule:

Task	Recommended resource
Inspect files / edit code	Login node
Small debugging run	Interactive CPU

Task	Recommended resource
NetCDF preprocessing / regridding	Interactive or batch CPU
Test PyTorch/JAX model	Interactive GPU
ACE / NeuralGCM experiment	Interactive GPU while developing
Long model run	Batch GPU
Large repeated preprocessing job	Batch CPU

The important distinction is that “**interactive**” vs. “**batch**” describes how you interact with the scheduled job, while “**CPU**” vs. “**GPU**” describes what type of compute hardware you request.

Therefore, all four combinations are possible:

```
Interactive CPU
Interactive GPU
Batch CPU
Batch GPU
```

Official NERSC Documentation

NERSC interactive-job documentation:

[Interactive Jobs — NERSC](#)

Perlmutter job-running documentation:

[Running Jobs on Perlmutter — NERSC](#)

NERSC beginner guide:

[NERSC Absolute Beginner's Guide](#)

Perlmutter CPU/GPU architecture:

[Perlmutter Architecture — NERSC](#)

Snow-17

For each generated weather realization, Snow-17 requires the following information:

Snow-17 requirement	What it is used for	What the intern should use	Status / notes
Precipitation $P(t)$	Supplies incoming water; Snow-17 determines how much contributes to snowfall/snowpack	Precipitation generated by the selected weather-generation model	Convert to the precipitation amount required at each Snow-17 forcing timestep. For our current setup, target

Snow-17 requirement	What it is used for	What the intern should use	Status / notes
			6-hourly precipitation amount in mm per 6 h. If the weather model outputs a rate, integrate over the 6-hour interval.
2-m air temperature $T(t)$	Controls rain/snow partition and temperature-index melt	Construct an adjusted 6-hourly temperature forcing using the matching CMIP6 2-m air temperature as the daily mean/background state , while preserving the 6-hourly temperature anomaly pattern generated by ACE-ERA5 or NeuralGCM . See the detailed construction procedure below.	Final Snow-17 temperature forcing should remain 6-hourly and in °C . Convert from K if needed.
Initial SWE	Represents snow already present when the Snow-17 integration begins	WUS-D3 d02 snow from the corresponding simulation and preceding completed day	Supply in mm water equivalent . We use simulated rather than observed SWE so initialization is physically consistent with the same simulated weather realization.
Other Snow-17 initial internal states	Initializes snow heat deficit, liquid water, temperature index, areal snow state, etc.	Must follow the documented initialization procedure of the chosen Snow-17 implementation	Not yet fully resolved. Do not invent arbitrary state values.
Terrain / elevation information	Needed if the implementation uses elevation zones, temperature adjustments, or basin elevation information	Use appropriate terrain/elevation data for the Sierra domain	Available from datasets such as USGS terrain products; exact implementation depends on the Snow-17 setup.
Spatial target	Defines what final SWE quantity should be reported	Final April-1 SWE must be area-averaged over 35°–42°N, 122.5°–118°W	Resolved. This is the same rectangular target used by the main CNN label.
SCF	Snowfall correction factor	Obtain from a calibrated Sierra Snow-17 configuration	Missing. Not an annually observed quantity.

Snow-17 requirement	What it is used for	What the intern should use	Status / notes
MFMAX	Maximum seasonal melt factor	Obtain from calibrated configuration	Missing.
MFMIN	Minimum seasonal melt factor	Obtain from calibrated configuration	Missing.
UADJ	Controls wind-related rain-on-snow melt contribution	Obtain from calibrated configuration	Missing.
PXTEMP	Temperature threshold used in precipitation-phase treatment	Obtain from calibrated configuration	Missing.
MBASE	Base temperature for melt calculations	Obtain from calibrated configuration	Missing.
NMF	Controls negative melt / snow heat-deficit evolution	Obtain from calibrated configuration	Missing.
TIPM	Antecedent snow-temperature index parameter	Obtain from calibrated configuration	Missing.
PLWHC	Liquid-water holding capacity of the snowpack	Obtain from calibrated configuration	Missing.
DAYGM	Daily ground-melt contribution	Obtain from calibrated configuration	Missing.
SI	Snow index / areal snow-cover scaling parameter	Obtain from calibrated configuration	Missing.
ADC	Areal depletion curve describing fractional snow-covered area as snowpack declines	Obtain the complete calibrated ADC curve	Missing.
Elevation-zone definitions and adjustments	Allows operational Snow-17 setups to represent elevation-dependent snow processes	Preserve the archived calibrated zone structure if the chosen Sierra configuration uses multiple zones	Not yet obtained.

Snow-17 requirement	What it is used for	What the intern should use	Status / notes
Complete calibrated Sierra parameter package	Makes the Snow-17 run scientifically defensible rather than relying on demo/default values	Preferred candidate: archived CNRFC NFDC1 Snow-17 configuration	Current major blocker. We know the calibration exists, but still need the complete authoritative configuration.

Constructing the 2-m Temperature Forcing

For every grid location and every day, construct the Snow-17 2-m temperature forcing using the following procedure.

1. Obtain the weather-model-generated 2-m air temperature

From the ACE-ERA5 or NeuralGCM realization, obtain the four 6-hourly 2-m air temperatures for that day:

$$T_{\text{ML},1}, T_{\text{ML},2}, T_{\text{ML},3}, T_{\text{ML},4}.$$

These temperatures must correspond to the **same location and same day** as the generated precipitation being supplied to Snow-17.

2. Calculate the generated daily mean and subdaily anomaly pattern

Calculate the weather-model-generated daily mean:

$$\mu_{\text{ML}} = \text{mean} (T_{\text{ML},1}, T_{\text{ML},2}, T_{\text{ML},3}, T_{\text{ML},4}).$$

Calculate the generated daily standard deviation:

$$\sigma_{\text{ML}} = \text{std} (T_{\text{ML},1}, T_{\text{ML},2}, T_{\text{ML},3}, T_{\text{ML},4}).$$

The standardized 6-hourly anomaly pattern is:

$$z_i = \frac{T_{\text{ML},i} - \mu_{\text{ML}}}{\sigma_{\text{ML}}}.$$

This captures the within-day temperature variation generated by ACE-ERA5 or NeuralGCM.

3. Obtain the matching CMIP6 2-m air temperature

Pull the corresponding CMIP6 **tas** field using the:

- **same parent CMIP6 model,**
- **same date,**
- **same spatial location.**

Calculate the corresponding CMIP6 daily-mean 2-m air temperature:

$$\mu_{\text{CMIP}}.$$

The CMIP6 temperature provides the background temperature associated with the original physical climate simulation.

4. Construct the adjusted 6-hourly temperature

Construct the Snow-17 temperature forcing by placing the generated subdaily variability around the CMIP6 daily mean:

$$T_{\text{adjusted},i} = \mu_{\text{CMIP}} + \sigma_{\text{ML}} z_i$$

Because

$$\sigma_{\text{ML}} z_i = T_{\text{ML},i} - \mu_{\text{ML}},$$

this can equivalently be implemented as:

$$T_{\text{adjusted},i} = \mu_{\text{CMIP}} + (T_{\text{ML},i} - \mu_{\text{ML}})$$

Therefore, **do not generate a new temperature pattern using only the standard deviation**. Preserve the actual 6-hourly temperature anomaly pattern produced by ACE-ERA5 or NeuralGCM, and shift that pattern so that it is centered on the corresponding CMIP6 daily mean.

The final adjusted temperature therefore has:

- the **daily mean/background temperature from the corresponding CMIP6 simulation**, and
- the **6-hourly temperature variation and standard deviation generated by ACE-ERA5 or NeuralGCM**.

Use this adjusted 6-hourly $T_{\text{adjusted}}(t)$, together with the generated precipitation $P(t)$, as the meteorological forcing for Snow-17.

The Lag-ensemble paper

<https://journals.ametsoc.org/view/journals/bams/107/5/BAMS-D-25-0178.1.xml>